

Ciudad << Atributos de instancia >> cp: entero poblacion: entero superficie: real << Constructores >> Ciudad(cod: entero) Ciudad(cod: entero, p: entero, s: float) << Comandos >> establecerPoblacion(pob: entero) establecerSuperficie(sup: real) actualizarPoblacion(cre: entero) << Consultas >> obtenerCP(): entero obtenerPoblacion(): entero obtenerSuperficie(): real obtenerDensidad(): real toString(): String equals(): boolean mayor(c: Ciudad): boolean	ColeccionCiudadesOrd <<atributos de instancia>> ciudades: [] Ciudad cant: entero << Constructores >> ColeccionCiudadesOrd(max: entero) <<Comandos>> insertar(c: Ciudad) eliminar(c: Ciudad) eliminarPorCant(Pob(c: int)) <<Consultas>> cant(Ciudades()): entero ... Las ciudades se mantienen ordenadas por cp en las primeras cant posiciones
---	--

```
class ColeccionCiudadesOrd{
    //Atributos de Instancia
    private Ciudad[] ciudades;
    private int cant;
    //Constructor
    public ColeccionCiudadesOrd(int max){
        ciudades = new Ciudad[max];
        cant = 0;
    }
    ...
}
```

```
<<atributos de instancia>>
ciudades [] Ciudad
cant: entero

<< Constructores>>
ColeccionCiudadesOrd(max:
entero)

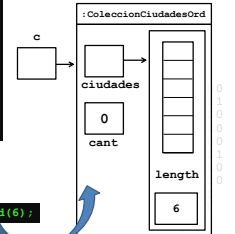
<<Comandos>>
insertar(c: Ciudad)
eliminar(c: Ciudad)
eliminarPorCantPob(cant: int)
...

<<Consultas>>
cantCiudades(): entero
...
```

```

Class ColeccionCiudadesOrd{
    //Atributos de Instancia
    private Ciudad[] ciudades;
    private int cant;
    //Constructor
    public ColeccionCiudadesOrd(int max){
        ciudades = new Ciudad[max];
        cant = 0;
    }
    -
}

```



Agrega una nueva ciudad c considerando la relación de orden establecida por la consulta mayor provista por la clase Ciudad. Requiere que la colección no esté llena, c esté ligada y la clase cliente haya verificado que no existe una ciudad con el mismo cp en la colección.

```
<<atributos de instancia>>
ciudades [] Ciudad
cant: entero

<< Constructores>>
ColeccionCiudadesOrd(max:
entero)
<<Comandos>>
insertar(c: Ciudad)
eliminar(c: Ciudad)
eliminarPorCantPob(cant: int)
...
<<Consultas>>
cantCiudades(): entero
...
```

- El servicio **insertar** tiene que funcionar considerando varios casos diferentes:
 - La colección está vacía
 - la ciudad:
 - Menor a todas
 - Mayor a todas
 - Mayor a la primera pero Menor que la última
- Asumimos que la clase cliente controla que la colección no está llena y la ciudad nueva no está en la colección.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

0 1 1 0 0
1 0 0 1 1
1 0
1 0
0 0
1 1
1 0
1 0
1
0 0
1


```

00
11
10
10
00
11
10
10
1
1 11
0 0
1

```

0	0
1	1
1	0
1	0
0	0
1	1
1	0
1	0
1	
1	

BÚSQUEDA BINARIA solución iterativa

Algoritmo pertenece

DE: c

ini $\leftarrow$ 0

fin $\leftarrow$ cantidad de ciudades-1

existe $\leftarrow$ falso

mientras no existe y ini $\leq$ fin

mitad $\leftarrow$ (ini+fin)/2

si T_{mitad} tiene la ciudad c buscada

existe $\leftarrow$ verdadero

sino

si T_{mitad} tiene una ciudad menor a la ciudad c

ini $\leftarrow$ mitad+1

sino

fin $\leftarrow$ mitad-1